CS236 Project Report: Phase 2

Team Members: Benedict Ignacio, Garett Hammerle
Due: November 13th, 2026

Spark Analysis

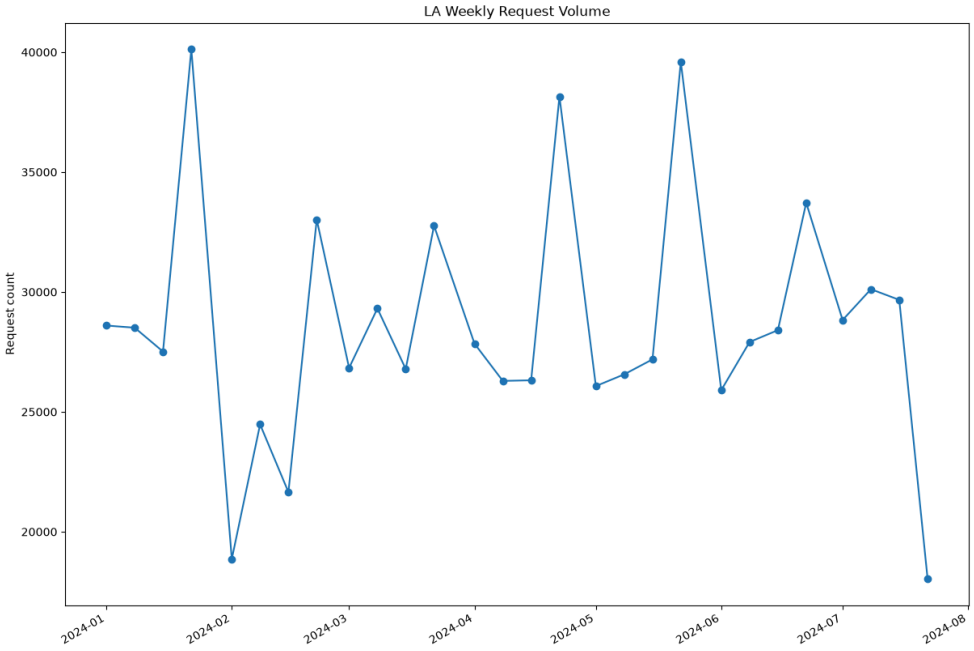
We wrote Pyspark scripts to perform the following analyses. All conclusions were made from print() and show() statements in the scripts, as well as charts generated using the matplotlib library. You can find these scripts in the same zip file as this document. We already performed some of this analysis in Phase 1, such as counting the top 10 zip codes and binning reports by date. We reuse some charts from that report on this one.

Weekly Request Volume

Implementation Details: For this analysis, we bucket the start dates of requests into time periods, particularly the day ranges 1-7, 8-14, 15-21, and 22-[end of month] for each month. For each row in the dataset, we truncate the ‘opened_date’ value and create a ‘day_of_month’ column that gives the day, and a ‘start_of_month’ column which truncates the ‘opened_date’ to the start of that month. For example, the date ‘2024-11-24’ will have a ‘day_of_month’ of ‘11’ and a ‘start_of_month’ of ‘2024-11-01’. We add a new ‘start’ column by looking at each ‘opened_date’ and checking which ‘day’ bucket it belongs to. Then, the value for the start column is just the ‘start_of_month’ value for that row plus the bucket’s offset. For example, our ‘2024-11-24’ opened_date will be bucketed into the 22-[end of month]. So the ‘start_date’ for this value is ‘2024-11-01’ + 21 days = ‘2024-11-22’. A similar technique goes for the ‘end’ column, which corresponds to the end date of the bucket. The last day of November is the 30th, so the ‘end’ column for ‘2024-11-24’ would be ‘2024-11-30’.

After bucketing every row into one of these weeks, each row now has a ‘start’ and ‘end’ column. We use groupBy() to group each unique bucket, then count the number of records in each group. We add this ‘count’ value to a new column.

We do this for each city. The LA bucketed dataframe has more rows than NYC because the date range is wider. After bucketing, we also looked at the week with the highest and lowest frequency of opened requests for each city. Below is the result of running our PySpark script and a chart to visualize it.



start	end	count
2024-01-01	2024-01-07	28600
2024-01-08	2024-01-14	28509
2024-01-15	2024-01-21	27513
2024-01-22	2024-01-31	40127
2024-02-01	2024-02-07	18860
2024-02-08	2024-02-14	24476
2024-02-15	2024-02-21	21653
2024-02-22	2024-02-29	33021
2024-03-01	2024-03-07	26832
2024-03-08	2024-03-14	29312
2024-03-15	2024-03-21	26787
2024-03-22	2024-03-31	32762
2024-04-01	2024-04-07	27845
2024-04-08	2024-04-14	26290
2024-04-15	2024-04-21	26316
2024-04-22	2024-04-30	38151
2024-05-01	2024-05-07	26074
2024-05-08	2024-05-14	26563
2024-05-15	2024-05-21	27188
2024-05-22	2024-05-31	39593

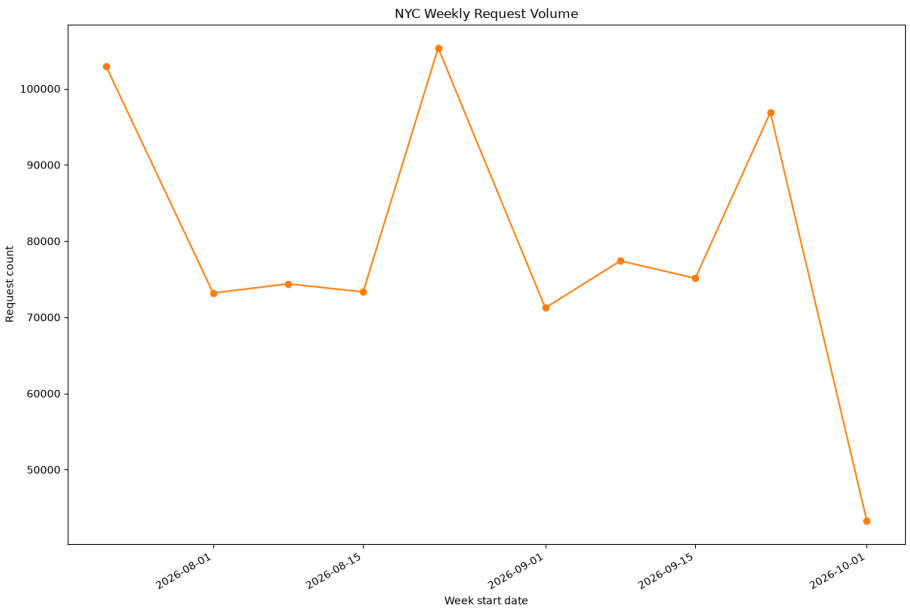
only showing top 20 rows

start	end	count
2024-07-22	2024-07-31	18044

start	end	count
2024-01-22	2024-01-31	40127

LA Request Volume

Here, we see that for LA, the request count distribution seems to be sporadic, though there are noticeable spikes for the buckets corresponding to 22-[end of month] for each month. This may be because the timeframe for these periods are more than the typical 7 days, lasting as long as 10 days. We see that the period with the highest number of requests is January 22-January 31, with 40,127 requests. Upon immediate observation, we also see that July 22-July 31 has the lowest number of requests at 18,044. However, this is because our date range for LA stops at July 24 (as demonstrated in Phase 1 of the project), which means that this period only lasts for 3 days, as opposed to at least 7 days. If we exclude this outlier, the period with the next smallest number for requests is February 1-February 7, with 18,860 requests.



start	end	count
2026-07-22	2026-07-31	102969
2026-08-01	2026-08-07	73179
2026-08-08	2026-08-14	74382
2026-08-15	2026-08-21	73322
2026-08-22	2026-08-31	105342
2026-09-01	2026-09-07	71228
2026-09-08	2026-09-14	77397
2026-09-15	2026-09-21	75118
2026-09-22	2026-09-30	96885
2026-10-01	2026-10-07	43297

start	end	count
2026-10-01	2026-10-07	43297

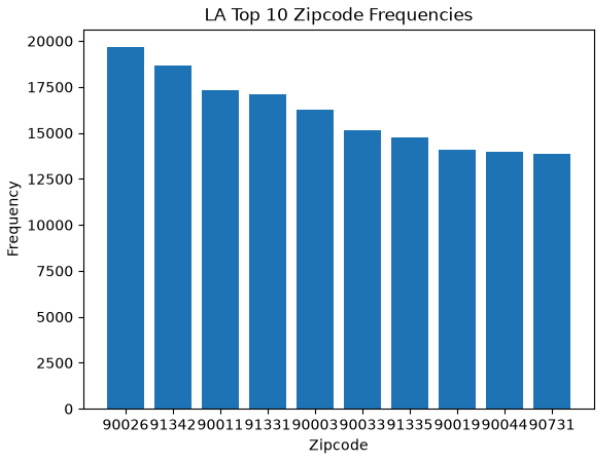
start	end	count
2026-08-22	2026-08-31	105342

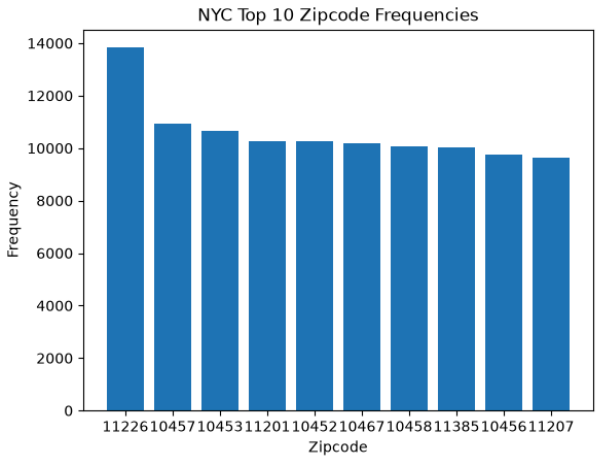
NYC Request Volume

The weekly request count distribution for NYC exhibits similar sporadic patterns as LA for the same reasons. We see that the period with the highest number of requests is August 22-August 31, with 105,342 requests, while the period with the lowest number of requests is October 1-October 7, with 43,297 requests. This is because the date range stops at October 5, meaning that this particular bucket only captures 5 days of requests rather than 7-10. Negating this outlier, the next lowest period corresponds to September 1-September 7, with 71,228 requests.

Request Hotspots

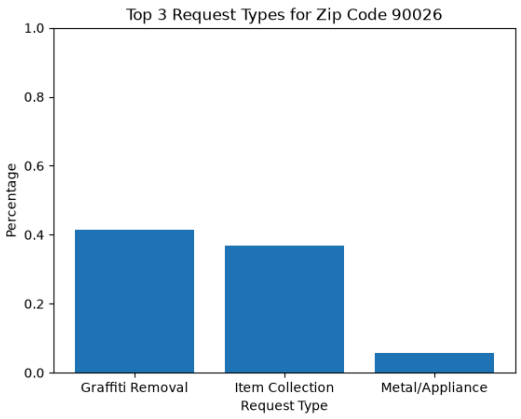
Implementation Details: To compute the frequency of request hotspots, we count distinct zip codes in each city's dataset. We put these counts in a new dataframe, sort it in descending order, and limit it to the first 10 rows (to get the top 10 most frequent zip codes). Next, we want to look at the number of requests for each type in each of these zip codes. For each of the top 10 zip codes, we filter the original dataset by that zip code, then count the frequency of each request type. We add this count as a column to a new dataframe, and sort it in descending order. We also compute the percentage of the total requests in this zip code for each of the types. We then limit this sorted dataframe to the top three rows to only consider the top three most frequent request types. Below are charts displaying the top 10 zip codes for LA and NYC (we also initially performed this analysis during Phase 1 of this project with our EDA):

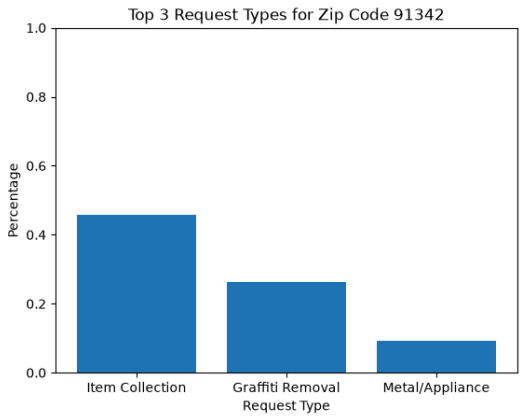


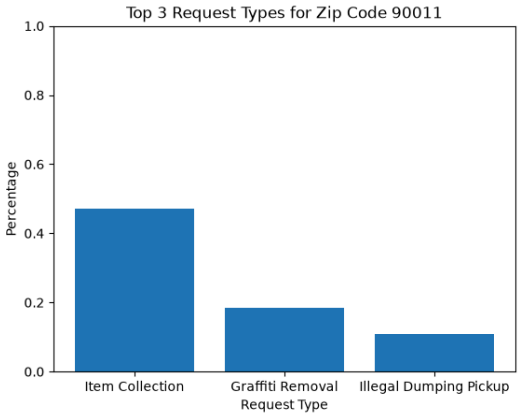


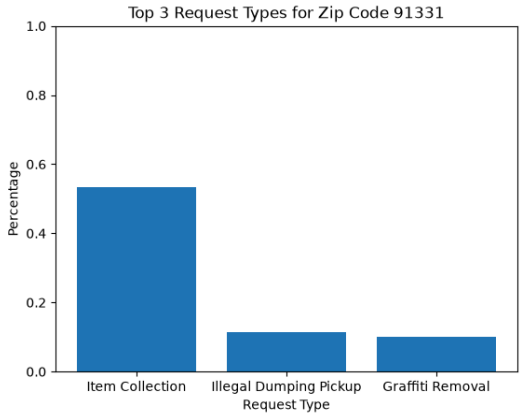
We also have the following relative frequency charts displaying the top three request types for the top 10 zip codes for each city:

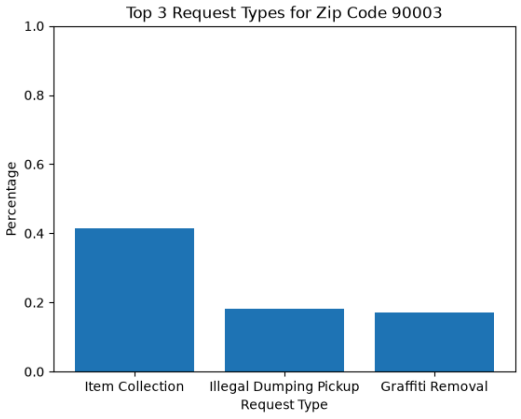
Top 3 Request Types for LA Zip Codes (by relative frequency)

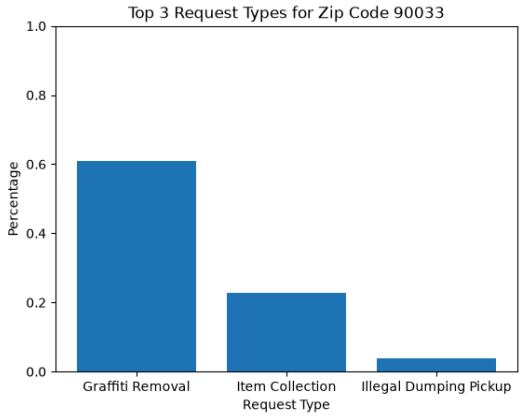

Top 3 Request Types for Zip Code 90026
Percentage
1.0
0.8
0.6
0.4
0.2
0.0
Graffiti Removal
Item Collection
Metal/Appliance
Request Type

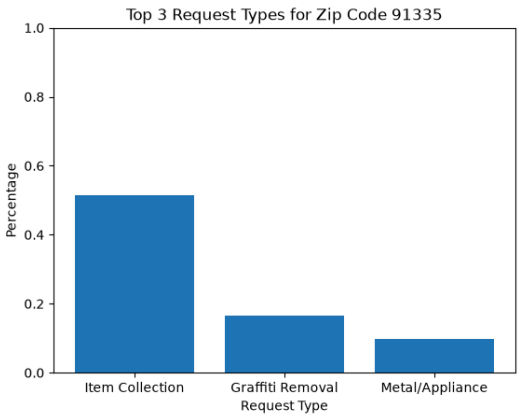

Top 3 Request Types for Zip Code 91342
Percentage
1.0
0.8
0.6
0.4
0.2
0.0
Item Collection
Graffiti Removal
Metal/Appliance
Request Type

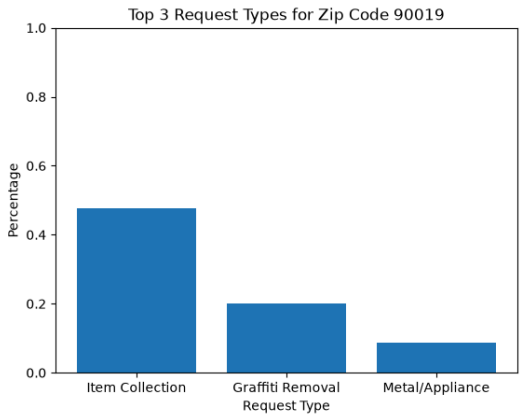

Top 3 Request Types for Zip Code 90011
Percentage
1.0
0.8
0.6
0.4
0.2
0.0
Item Collection
Graffiti Removal
Illegal Dumping Pickup
Request Type

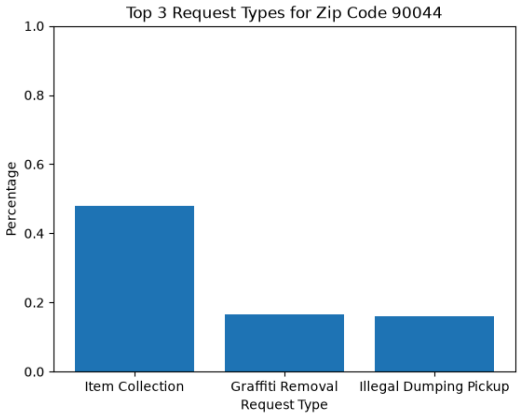

Top 3 Request Types for Zip Code 91331
Percentage
1.0
0.8
0.6
0.4
0.2
0.0
Item Collection
Illegal Dumping Pickup
Graffiti Removal
Request Type

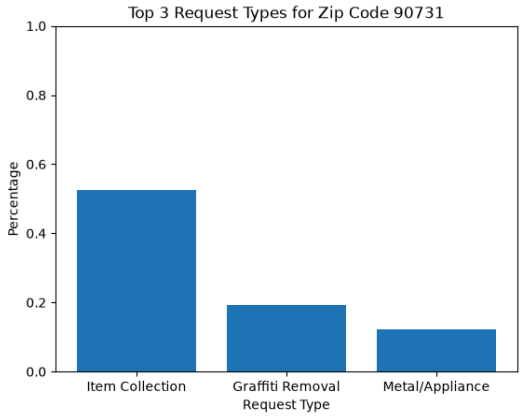

Top 3 Request Types for Zip Code 90003
Percentage
1.0
0.8
0.6
0.4
0.2
0.0
Item Collection
Illegal Dumping Pickup
Graffiti Removal
Request Type


Top 3 Request Types for Zip Code 90033
Percentage
1.0
0.8
0.6
0.4
0.2
0.0
Graffiti Removal
Item Collection
Illegal Dumping Pickup
Request Type

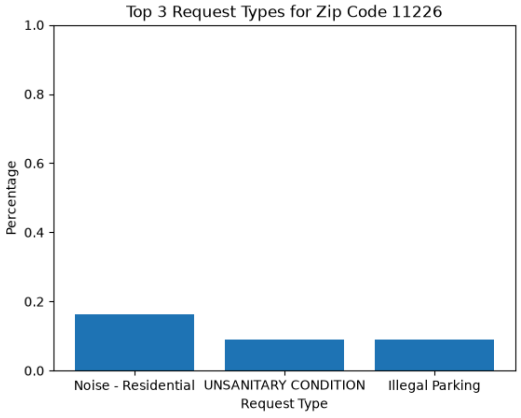


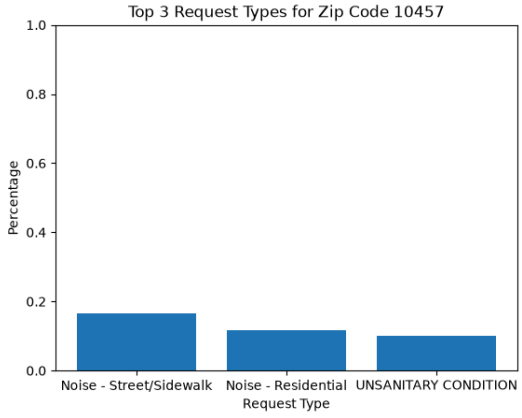


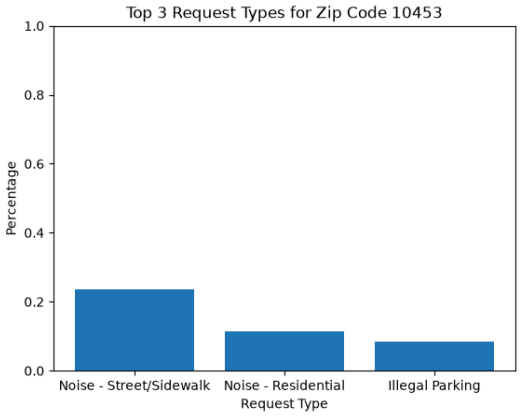


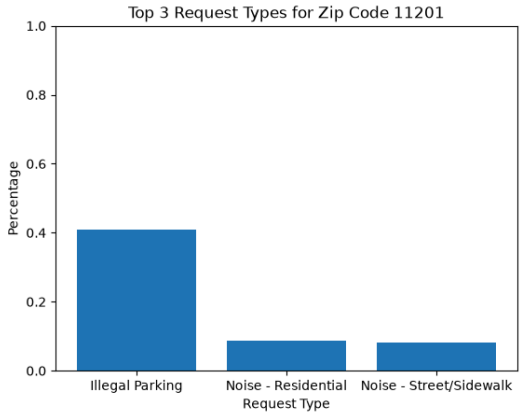


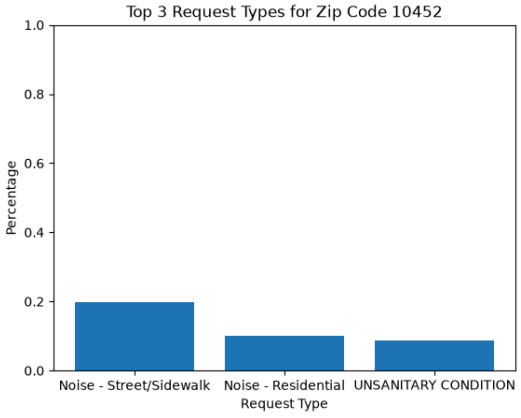
Top 3 Request Types for NYC Zip Codes (by relative frequency)

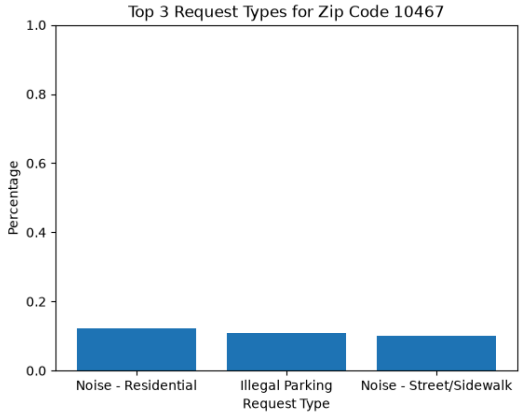


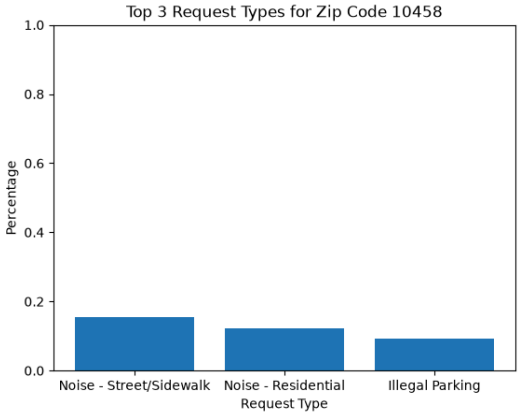


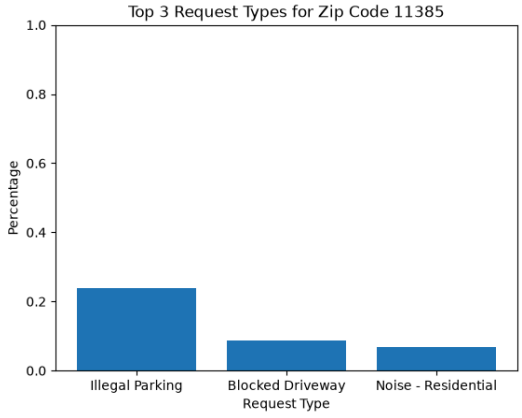

Top 3 Request Types for Zip Code 10453
Percentage
1.0
0.8
0.6
0.4
0.2
0.0
Noise - Street/Sidewalk
Noise - Residential
Illegal Parking
Request Type

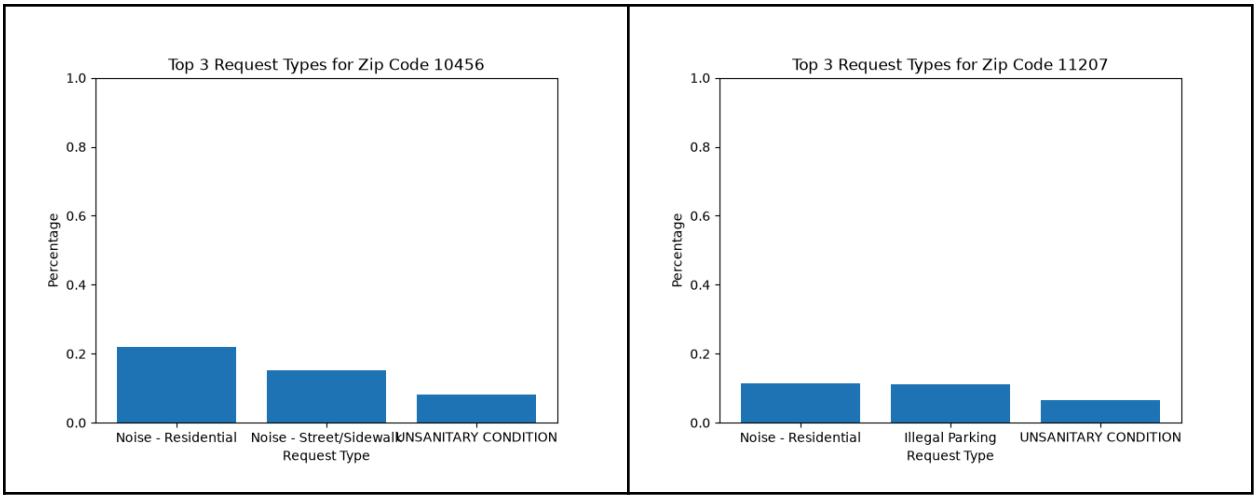

Top 3 Request Types for Zip Code 11201
Percentage
1.0
0.8
0.6
0.4
0.2
0.0
Illegal Parking
Noise - Residential
Noise - Street/Sidewalk
Request Type


Top 3 Request Types for Zip Code 10452
Percentage
1.0
0.8
0.6
0.4
0.2
0.0
Noise - Street/Sidewalk
Noise - Residential
UNSANITARY CONDITION
Request Type


Top 3 Request Types for Zip Code 10467
Percentage
1.0
0.8
0.6
0.4
0.2
0.0
Noise - Residential
Illegal Parking
Noise - Street/Sidewalk
Request Type


Top 3 Request Types for Zip Code 10458
Percentage
1.0
0.8
0.6
0.4
0.2
0.0
Noise - Street/Sidewalk
Noise - Residential
Illegal Parking
Request Type


Top 3 Request Types for Zip Code 11385
Percentage
1.0
0.8
0.6
0.4
0.2
0.0
Illegal Parking
Blocked Driveway
Noise - Residential
Request Type



Our results indicate that New York has more variety in its request types. For all of the top ten zip codes in NYC, the most common request type is usually never more than 20% of all requests. The one exception to this is in the 11201 zip code, where illegal parking makes up over 40% of the reports. The most common request type is almost always noise complaints, with the rest being illegal parking reports.

LA has less variety, with the most frequent request type always making up over 40% of the requests in that zip code. Common request types for LA are graffiti removal and item collection. Item collection seems to dominate more, but certain zip codes have a high proportion of graffiti removal compared to others.

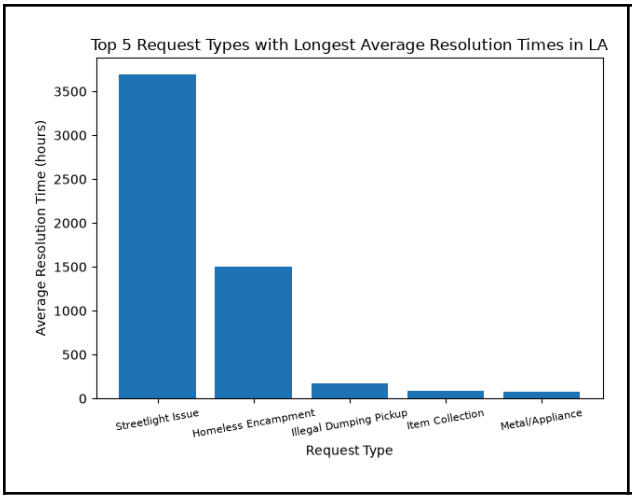
(Bonus) Resolution Performance

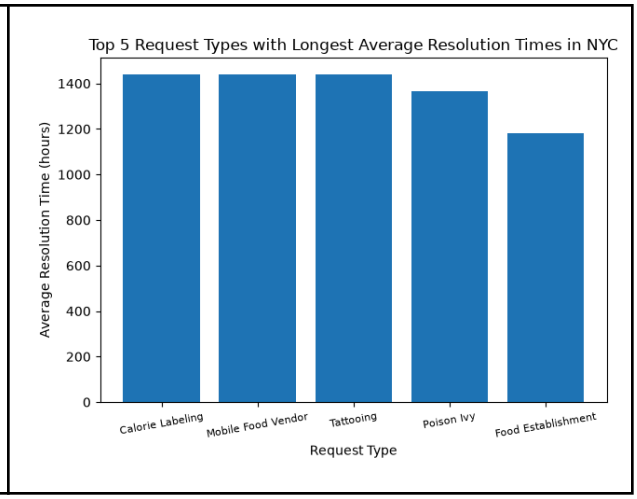
Implementation Details: Using the unified dataset from Phase 1, we create a separate dataframe that filters the dataset down to only the rows that have a "Closed" status. Then we further split into two separate dataframes per city. This is where our 'request_time' column (calculated as closed date minus opened date) from Phase 1 comes in handy. We use this precalculated value to determine the average and median resolution time across each city.

Then, for each city, we create a separate dataframe instance that identifies and filters the top 5 request types by longest average resolution time. The raw statistics and bar charts displaying the data are shown below:

```
LA Average Resolution Time: 229.37908878263545 hours
LA Median Resolution Time: 46.52722222222222 hours
+--------------------+--------------------+
|        request_type|avg_resolution_time|
+--------------------+--------------------+
|   Streetlight Issue|    3696.3345520596|
| Homeless Encampment| 1505.2000931311445|
|Illegal Dumping P...| 174.00548320589175|
|     Item Collection|  84.69707748727751|
|     Metal/Appliance|  75.19456691852432|
+--------------------+--------------------+
```

```
NYC Average Resolution Time: 89.269071900255 hours
NYC Median Resolution Time: 4.811111111111111 hours
+------------------+--------------------+
|      request_type|avg_resolution_time|
+------------------+--------------------+
|  Calorie Labeling| 1440.2944444444445|
|Mobile Food Vendor| 1440.2628304284676|
|         Tattooing| 1440.2355555555557|
|        Poison Ivy| 1366.8591666666664|
|Food Establishment| 1181.0495615956463|
+------------------+--------------------+
```





Looking at overall statistics, we see that LA takes longer to resolve 311 issues, both in median and average resolution times. The average resolution time for LA is 229 hours, as opposed to NYC's 89 hours. For median resolution times, LA takes 47 hours, whereas NYC takes only 5 hours. Here, we can say with confidence that NYC is more efficient in resolving and closing 311 cases than LA.

When we take a closer look at the top five request types with the longest average resolution times, we find varying distributions for each city. For LA, the top five request types are streetlight issues, homeless encampments, illegal dumping, item collection, and metal appliance issues. By far the request type that takes the longest to close are streetlight issues (3,696 hours, or 154 days on average). For NYC, the top five request types are calorie labeling, mobile food vendors, tattooing, poison ivy, and food establishment, with the top three averaging around 1,440 hours (60 days).

(Bonus) Status and Submission Channels

Implementation Details: Using the unified dataset, we split into two dataframes filtered by city. From there, we calculate the absolute and relative frequency of requests in each status category (which is already standardized from Phase 1) per city. Then, we do the same for each submission channel category (listed under 'source' column) per city. Note that the bar plots only list the top five statuses and submission channels for the sake of brevity.

LA Request Types	NYC Request Types

status	count	percentage
Open	15578	0.01949504114131965
Cancelled	34521	0.04320120138910615
Pending	17970	0.02248850233082001
Closed	729910	0.9134436692425617
Assigned	1096	0.001371585896192...

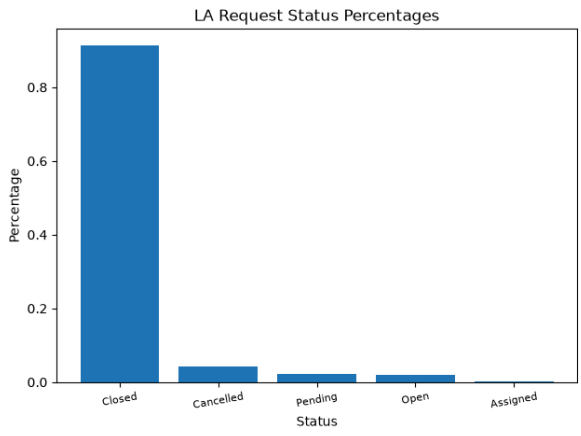
status	count	percentage
Open	36568	0.04610657417108908
In Progress	54086	0.06819405410789553
Unspecified	52	6.556393176812054E-5
Pending	327	4.122962632341427E-4
Closed	698849	0.8811401567734476
Assigned	3223	0.004063702924781...
Started	14	1.765182778372476...

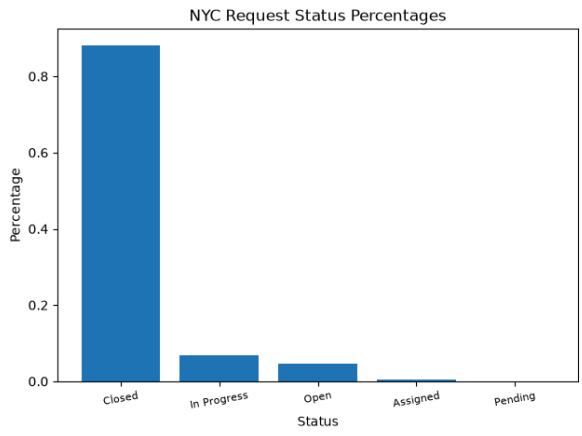
LA Submission Channels

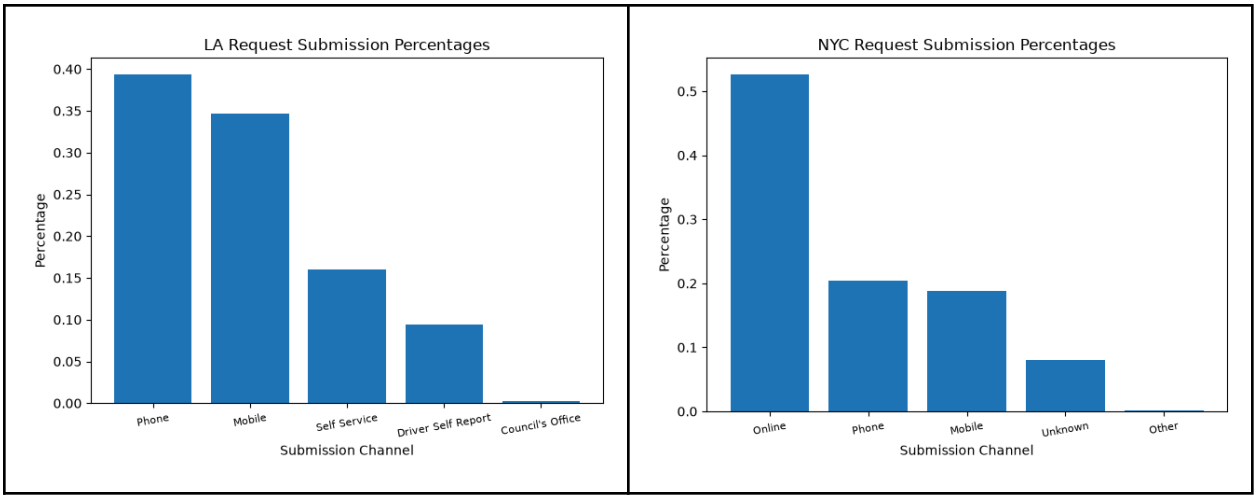
source	count	percentage
Council's Office	1940	0.002427807152019...
Queue Initiated C...	1	1.251446985577073...
Self Service	128299	0.16055939680255296
Phone	314581	0.39368144416982137
Fax	4	5.005787942308294E-6
Social	51	6.382379626443075E-5
Driver Self Report	75496	0.09447924162312674
Online	1523	0.001905953759033883
Mobile	277173	0.3468673153333542
Letter	1	1.251446985577073...
Walk-in	4	5.005787942308294E-6
City Attorney	1	1.251446985577073...
Radio	1	1.251446985577073...

NYC Submission Channels

source	count	percentage
Phone	162107	0.2043917747525907
Unknown	63879	0.080541507642611
Online	417000	0.525772298986659
Mobile	149344	0.1882996120380422
Other	789	9.948065800970598E-4







From the charts, we can easily tell that for both LA and NYC, the vast majority of 311 cases have been closed, with 91.3% (729,910 requests) and 88.1% (698,849 requests) respectively. However, when we take a look at how people submitted 311 requests, we see that for LA, the most common source of requests was through the phone, taking up 39.4% (314,581 requests) of the total requests. For NYC, the most common source of requests was online, taking up 52.6% (417,000 requests) total.

Database Population

Designing the Schema

In this part, we load both our cleaned datasets and our merged dataset into a Postgres database. First, we designed the Postgres schema for each of the datasets. These match exactly the schemas in the first report, but we also add nullability and primary key restrictions on certain columns. Below are the schemas with their restrictions:

Merged Dataset Schema (and data type):	**Restriction**
id (BIGINT)	PRIMARY KEY
opened_date (TIMESTAMP WITH TIME ZONE)	NOT NULL
closed_date (TIMESTAMP WITH TIME ZONE)	
status (TEXT)	NOT NULL
request type (TEXT)	NOT NULL
source (TEXT)	NOT NULL
address (TEXT)	
zip_code (TEXT)	NOT NULL
service_date (TIMESTAMP WITH TIME ZONE)	
council_district (INTEGER)	
action_taken (TEXT)	
neighborhood_council_name (TEXT)	
police_precinct (TEXT)	
latitude (DOUBLE PRECISION)	
longitude (DOUBLE PRECISION)	

location (TEXT)	
request_duration (DOUBLE PRECISION)	
response_time (DOUBLE PRECISION)	
resolve_time (DOUBLE PRECISION)	
source_city (TEXT)	NOT NULL
original_key (TEXT)	NOT NULL

LA Schema (and data type)		**NYC Schema (and data type)**	
original_key (TEXT)	NOT NULL	original_key - record identification (TEXT)	NOT NULL
created_date (TIMESTAMP WITH TIME ZONE)	NOT NULL	created_date (TIMESTAMP WITH TIME ZONE)	NOT NUL
updated_date (TIMESTAMP WITH TIME ZONE)		closed_date (TIMESTAMP WITH TIME ZONE)	
action_taken (TEXT)		agency (TEXT)	
owner (TEXT)		agency_name (TEXT)	
request_type (TEXT)	NOT NULL	request_type (TEXT)	NOT NULL
status (TEXT)	NOT NULL	descriptor (TEXT)	
source (TEXT)	NOT NULL	descriptor (TEXT)	
created_by_user_org (TEXT)		location_type (TEXT)	
anonymous (TEXT)		zip_code (TEXT)	NOT NULL
assign to (TEXT)		address (TEXT)	
service_date (TIMESTAMP WITH TIME ZONE)		street_name (TEXT)	
closed_date (TIMESTAMP WITH TIME ZONE)		cross_street_1 (TEXT)	
address_verified (TEXT)		cross_stree_2 (TEXT)	
approx_address (TEXT)		intersection_street_1 (TEXT)	
address (TEXT)		intersection_street_2 (TEXT)	
house_number (INTEGER)		address_type (TEXT)	
direction (TEXT)		city (TEXT)	
street_name (TEXT)		landmark (TEXT)	
suffix (TEXT)		status (TEXT)	NOT NULL
zip_code (TEXT)	NOT NULL	resolution_description (string)	
latitude (DOUBLE PRECISION)		service_date (TIMESTAMP WITH TIME ZONE)	
longitude (DOUBLE PRECISION)		community_board (TEXT)	
location (TEXT)		council_district (INTEGER)	
tbm_page (INTEGER)		police_precinct (TEXT)	
tbm_column (TEXT)		bbl (BIGINT)	
tbm_row (INTEGER)		borough (string)	
apc - automatic passenger		state_plane_x	

counter (TEXT)		(INTEGER)	
cd - council district (INTEGER)		state_plane_y (INTEGER)	
cd_member (TEXT)		source (TEXT)	NOT NULL
nc (INTEGER)		park_facility_name (TEXT)	
nc_name (TEXT)		park_borough (TEXT)	
police_precinct (TEXT)		latitude (DOUBLE PRECISION)	
		longitude (DOUBLE PRECISION)	
		location (TEXT)	

We had to make some decisions about attribute data types when designing the schemas for these tables. For instance, in the NYC dataset the original_key attribute (the unique identifier) is an integer, but in the LA dataset this attribute is a string. We wanted to enable the user to find the same record in the merged and cleaned datasets, because there's more information about a given record in its original dataset. We can't use the original_key as-is, because LA and NYC have different formats for their keys and the data types wouldn't match.

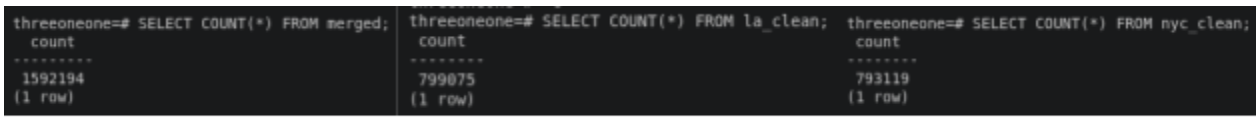
Our solution is to cast the original_key to a string (for each city) and write that to the merged dataset. This should let the user go backwards from the merged dataset and find any given record in its source dataset. The downside is that there isn't any index on the cleaned datasets because their key is a string. We think the benefits outweigh the downsides.

Every column we give a non-nullity constraint to has zero null values once the cleaning stage from phase 1 is done. Some of them are columns we add when merging, such as id and source_city. Others are columns like zip_code and request_type, which we previously intentionally dropped null values of.

Populating the Database

To write the data (which have been in CSV files since Phase 1) to the live Postgres database, we used an instance of Postgres running inside a Docker container. We wrote a Docker Compose YAML, Dockerfile, and SQL script to make the process of creating the container and database easier. In the Dockerfile, we pull a Postgres image, then run the SQL script (init.sql) which creates the tables with our predefined schemas. To actually populate the database with our CSVs from the previous part, we have a PySpark script (postgres.py) that reads the CSVs, sets the proper credentials and URL to connect to the Postgres container, then writes to the tables using JDBC.

To verify that all records are present in the Postgres database, we connect to the container and run commands to count the number of rows in each table. Below is the output of these commands.



SQL commands run from the container which count the number of rows in each table. They all match the counts from Phase 1.